
Spectral Element Method for Acoustic Wave Propagation in 2D

16214-01 - Project: Numerical Methods for Partial Differential Equations | Spring Semester, 2026

University of Basel, Department of Mathematics and Computer Science
Prof. Dr. M. J. Grote | Dr. Romain Mottier

Deadline for submission: Friday, June 5, 2026

Abstract

The objective of this project is to implement an arbitrary order spectral element method to solve two-dimensional linear acoustic wave propagation. Don't be intimidated by the length of the project — it mostly looks long because it's explaining things in detail, not because it's secretly difficult ☺.

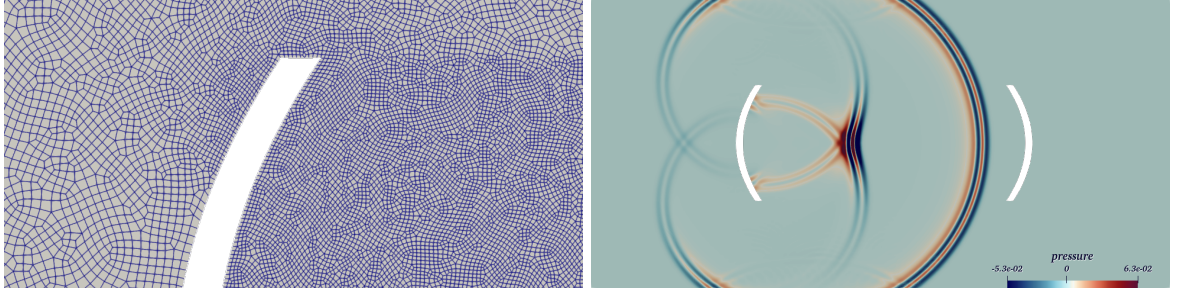


Figure 1: **Left frame:** Zoom on an unstructured quadrangular mesh. **Right frame:** Propagation of a Ricker wavelet in a computational domain with two reflectors.

Table of Contents

1	Preliminaries	1
2	Exercise 1: Test case with analytical solution	4
3	Exercise 2: Reflection of a Ricker wavelet between two parabolic antennas	9

1 Preliminaries

1.1 Notation

Let Ω be a polyhedral domain (open, bounded, connected, Lipschitz set) in $\mathbb{R}^d$, with $d \in \{2, 3\}$ and $\mathcal{T}_h$ denote a conforming partition of Ω into non-overlapping elements. For any element $T \in \mathcal{T}_h$, the superscript (T) is used to denote quantities defined locally on T .

Let $J := (0, T_f)$ denotes the time interval of interest, where $T_f > 0$ represents the final time, and let Ω be a polyhedral domain (open, bounded, connected, Lipschitz set) in $\mathbb{R}^d$, with $d \in \{2, 3\}$. For a bounded and uniformly positive weight $\kappa \in L^\infty(\Omega)$, we define the κ -weighted L^2 -inner products as

$$(u, v)_{L^2(\kappa; \Omega)} := \int_{\Omega} \kappa uv \, dx.$$

1.2 Model problem

Acoustic wave propagation is modeled by the evolution of its scalar pressure field p [Pa], which is described by the following, linear partial differential equation in $J \times \Omega$:

$$\frac{1}{\kappa} \partial_{tt} p - \nabla \cdot \left(\frac{1}{\rho} \nabla p \right) = f, \quad (1.1)$$

where $\rho \left[\frac{\text{kg}}{\text{m}^3} \right]$ denotes the fluid density, $\kappa [\text{Pa}]$ is the bulk modulus of the fluid, and $f [\text{s}^{-2}]$ represents the source term. The material parameters are assumed to be positive and piecewise constant in Ω . Homogeneous Neumann boundary conditions, which model perfectly rigid boundaries, are imposed:

$$\left. \frac{\partial p}{\partial \mathbf{n}} \right|_{\partial \Omega} = 0.$$

The propagation speed of acoustic waves is defined by $c_p := \sqrt{\frac{\kappa}{\rho}} \left[\frac{\text{m}}{\text{s}} \right]$. The problem is complemented with the initial conditions $p(0) = p_0$, $\partial_t p(0) = v_0$.

Question 1. Derive the weak formulation of the linear acoustic wave equation (1.1) with homogeneous neuman boundary conditions.

1.3 Reminders on Finite Element Methods

In order to approximate the acoustic wave equation (1.1), the idea is first to discretize the spatial variable using a finite element method (FEM), while keeping time continuous.

Finite Element Method. On each element $T \in \mathcal{T}_h$, we introduce a local polynomial space $\mathbb{P}^k(T)$ of degree at most k . The global finite element space is then defined as

$$V_h := \{v_h \in V : v_h|_T \in \mathbb{P}^k(T) \quad \forall T \in \mathcal{T}_h\}.$$

The space V_h is spanned by a finite set of globally continuous basis functions $\varphi_{i=1}^{N_h}$, associated with the mesh degrees of freedom (e.g., nodal values in the case of Lagrange elements, or coefficients for modal bases). In the case of nodal Lagrange elements, these basis functions satisfy the interpolation property

$$\varphi_i(\mathbf{x}_j) = \delta_{ij}, \quad \forall i, j \in \{1, \dots, N_h\},$$

where $\mathbf{x}_j \in \mathbb{R}^d$ denotes the j -th mesh node. Any semi-discrete approximation $p_h(t) \in V_h$ can therefore be written as

$$p_h(t) = \sum_{i=1}^{N_h} P_i(t) \varphi_i,$$

where the coefficients $P_i(t)$ denote the time-dependent degrees of freedom. This representation corresponds to the classical **Galerkin approximation**. Substituting this expansion into the weak formulation of (1.1) yields the semi-discrete problem: Find $p_h(t) \in V_h$ such that

$$(\partial_{tt} p_h(t), q_h)_{L^2(\frac{1}{\kappa}; \Omega)} + a_h(p_h, q_h) = (f, q_h)_{L^2(\Omega)} \quad \forall q_h \in V_h.$$

Choosing the test functions successively $q_h = \varphi_j$, for $j = 1, \dots, N_h$, leads to the matrix system

$$\mathbf{M}\ddot{\mathbf{P}}(t) + \mathbf{K}\mathbf{P}(t) = \mathbf{F}(t), \tag{1.2}$$

where $M_{ij} = (\varphi_i, \varphi_j)_{L^2(\frac{1}{\kappa}; \Omega)}$, $K_{ij} = a_h(\varphi_i, \varphi_j)$, $F_j(t) = (f, \varphi_j)_{L^2(\Omega)}$, $i, j = 1, \dots, N_h$.

Importance of the algebraic properties of the matrix system for time integration. The algebraic structure of the matrices in (1.2) play a crucial role in the efficiency of the time integration procedure.

1. **Implicit time integration.** Implicit schemes, require the solution of a large linear system at each time step. Even though $\mathbf{M}$ and $\mathbf{K}$ are sparse, the resulting matrix must be factorized or iteratively solved at every time step. For large-scale wave propagation problems in two or three space dimensions, this leads to a significant computational cost and memory requirements.

For these reasons, implicit methods are generally not well-suited for large-scale wave propagation problems.

2. **Explicit time integration.** Explicit schemes are particularly attractive for time integration of hyperbolic problems. When a lumped mass matrix is used, they avoid solving linear systems and instead update the solution through matrix-vector products. The computational cost per time step is therefore significantly reduced compared to implicit schemes.

However, explicit schemes are conditionally stable and must satisfy a Courant–Friedrichs–Lewy (CFL) condition of the form

$$\Delta t \leq C \frac{h}{c_p},$$

where h denotes the mesh size and c_p the wave speed. Despite this restriction, explicit schemes remain very efficient in practice for wave propagation problems. Thus, a crucial requirement for the efficiency of explicit schemes is the ability to compute $\mathbf{M}^{-1}$ at low cost.

For this reason, it is highly desirable that the mass matrix be diagonal (or block-diagonal). In that case, $\mathbf{M}^{-1}$ is trivial to compute, since it only involves component-wise (or small block-wise) divisions. This property dramatically reduces the computational cost and enables fully explicit time stepping.

Standard finite elements typically produce a consistent mass matrix that is sparse but not diagonal. To simplify explicit time integration, mass lumping techniques can approximate $\mathbf{M}$ by a diagonal matrix, though this may slightly reduce accuracy. On triangular meshes, these strategies can be applied to linear ($\mathbb{P}^1$) and quadratic ($\mathbb{P}^2$) elements, but higher-order elements are increasingly complicated and expensive.

In contrast, high-order spectral element methods combined with Gauss–Lobatto–Legendre (GLL) quadrature on quadrilaterals or hexahedra naturally yield diagonal mass matrices. This property makes these methods particularly well suited for fully explicit time integration of wave equations.

Spectral Element Method The Spectral Element Method (SEM) can be regarded as a high-order variant of FEM, where the choice of basis functions and quadrature nodes is optimized. In SEM, the computational domain is typically partitioned into quadrilateral elements in 2D or hexahedral elements in 3D.

By taking tensor products of the one-dimensional GLL Lagrange polynomials in each spatial direction, we construct the basis functions for 2D or 3D elements.

In the 2D case, on each element T , the local approximation space is defined as

$$\mathbb{Q}^k(T) := \text{span} \left\{ \ell_i(x) \ell_j(y) : i, j = 0, \dots, k \right\},$$

where $\ell_i(x)$ and $\ell_j(y)$ are one-dimensional Lagrange polynomials of degree k associated with the GLL nodes in the x and y directions, respectively. These nodes correspond to the extrema of the Legendre polynomials, including the endpoints, which ensures exact interpolation at the element boundaries.

The resulting global finite element space is then

$$V_h := \left\{ v_h \in V : v_h|_T \in \mathbb{Q}^k(T) \ \forall T \in \mathcal{T}_h \right\}.$$

A distinctive feature of SEM is **the use of GLL nodes both to define the basis functions and the quadrature nodes**. This choice makes the mass matrix purely diagonal, making SEM particularly efficient for time-dependent wave propagation problems and enabling fully explicit time-stepping schemes.

Finally, the accuracy of SEM increases rapidly with the polynomial degree k , provided the solution is smooth, leading to spectral (exponential) convergence in the polynomial order. SEM is therefore widely employed in applications such as seismic wave simulations.

Incompatibility of SEM with triangles and tetrahedra. To perform mass-lumping on triangular elements, one requires a quadrature rule that is exact for polynomials of degree $2k - 1$ and whose points coincide with the interpolation nodes. However, straightforward computations show that such a rule inevitably produces negative weights or zeros on the diagonal of the mass matrix. Consequently, some eigenvalues of the discrete operator may become negative, leading to exponentially growing solutions. While alternative quadrature rules can guarantee positive weights, they generally fail to preserve the exactness needed for high-order convergence.

To summarise, the computational efficiency of the spectral element method relies on two key properties;

1. the use of GLL points on quads/hexas as interpolation and quadrature nodes renders the mass matrix diagonal, so that its action reduces to inexpensive pointwise scaling and no global assembly or inversion is required;
2. the tensor-product structure of the GLL-based high-order basis functions allows elemental operators to be evaluated very efficiently through sequences of one-dimensional operations, which are particularly well suited for vectorization and parallel execution.

Together, these two features enable the spectral element method to be implemented in a fully matrix-free finite element framework, where discrete operators are applied on the fly with optimal computational efficiency.

2 Exercise 1: Test case with analytical solution

We consider the 2D acoustic wave equation (1.1), on the domain $\Omega = [0, L_x] \times [0, L_y]$. To verify the following implementation, we construct an exact solution that satisfies both the PDE and the boundary condition, then derive the corresponding source term f by direct substitution. The chosen manufactured solution is:

$$p(x, y, t) = \cos(k_x x) \cos(k_y y) (1 + t), \quad (2.1)$$

with integer mode numbers $n_x = n_y = 2$ and spatial wavenumbers

$$k_x = \frac{n_x \pi}{L_x}, \quad k_y = \frac{n_y \pi}{L_y}, \quad K^2 = k_x^2 + k_y^2.$$

The cosine functions satisfy $\partial_x \cos(k_x x)|_{x=0, L_x} = 0$ and $\partial_y \cos(k_y y)|_{y=0, L_y} = 0$ for any integer mode number, so the Neumann condition (1.2) is automatically satisfied on all four sides. Substituting (2.1) into (1.1) yields the purely spatial source:

$$f(x, y, t) = \frac{K^2}{\rho} \cos(k_x x) \cos(k_y y) (1 + t). \quad (2.2)$$

The initial conditions are set to the exact values at $t = 0$:

$$p(x, y, 0) = \cos(k_x x) \cos(k_y y), \quad v(x, y, 0) := \left. \frac{\partial p}{\partial t} \right|_{t=0} = \cos(k_x x) \cos(k_y y).$$

The goal of the exercise is to measure the error L^2 and H^1 errors at the final time T_f .

2.1 Task 1: Construction of the GLL nodes, weights and derivative matrix for a reference element

In this project, we consider SEM of arbitrary polynomial order k , and the implementation must therefore support any given order. For this purpose, the GLL nodes for orders ranging from 2 to 19 are provided for a reference element in the folder `gll_xwh` on ADAM.

The function `GetGLL(nGLL)`, where $\text{nGLL} = k + 1$ denotes the number of GLL points, reads the corresponding pre-tabulated file and returns three quantities:

1. the coordinates of the GLL nodes on the reference segment $[-1, 1]$,
2. the associated quadrature weights $\{w_i\}_{i=1}^n$,
3. and the $n \times n$ spectral derivative matrix $H_{ij} = \ell'_i(\xi_j)$, whose (i, j) entry is the derivative of the i -th Lagrange basis function evaluated at the j -th GLL node.

Call `[xi_gll, w_gll, H] = GetGLL(nGLL)` to construct the SEM discretization on the reference element.

2.2 Task 2: Mesh construction and global connectivity

Two functions are provided to build the geometric and topological description of the spectral element mesh:

- `build_cartesian_mesh`: constructs the physical mesh, metric terms, and global connectivity,
- `build_sem_discretization`: builds the global connectivity array enforcing C^0 -continuity (called internally by the first function).

Call `build_cartesian_mesh(Lx, Ly, Nel_x, Nel_y, nGLL, xi_gll)` to construct a tensor-product mesh of $\text{Nel}_x \times \text{Nel}_y$ elements on the domain $[0, L_x] \times [0, L_y]$, each element containing a $\text{nGLL} \times \text{nGLL}$ grid of GLL nodes. The function returns:

- `iglob(i, j, e)`: global index of local node (i, j) in element e ,
- `x, y`: physical coordinates of all global nodes,
- `jac`: Jacobian determinant at each GLL node,
- `dxdxi_all, dxdetax_all, dydxi_all, dydeta_all`: metric terms,
- `n_elem, nglob`: total number of elements and global nodes.

2.3 Task 3: Time step selection

Immediately after the mesh is built and the space discretisation established, we choose a suitable time step. The leapfrog scheme used for time integration (see Task 5) is conditionally stable and requires

$$\Delta t \leq \text{CFL} \frac{\Delta h_{\min}}{c}, \quad (2.3)$$

where $\Delta h_{\min}$ is the minimum Euclidean distance between any two locally adjacent GLL nodes along the ξ -direction (i.e. nodes (i, j) and $(i + 1, j)$ within the same element), taken over all elements and all rows j .

Implement the function `get_stable_dt(iglob, x, y, n_elem, nGLL, CFL, c, TMAX)` as follows. Loop over all elements e , all rows $j \in \{1, \dots, \text{nGLL}\}$, and all adjacent pairs $i \in \{1, \dots, \text{nGLL} - 1\}$. For each pair, retrieve the global indices

$$n_1 = \text{iglob}(i, j, e), \quad n_2 = \text{iglob}(i + 1, j, e),$$

and compute the Euclidean distance between the corresponding physical nodes:

$$d = \sqrt{(x(n_2) - x(n_1))^2 + (y(n_2) - y(n_1))^2}.$$

Track the global minimum $\Delta h_{\min}$ over all such pairs. The time step is then set to:

$$N_T = \left\lceil \frac{T_f}{\text{CFL} \Delta h_{\min}/c} \right\rceil, \quad \Delta t = \frac{T_f}{N_T}.$$

As the GLL nodes cluster near the element boundaries, $\Delta h_{\min} \sim h/k^2$ and $h \sim 1/N_{\text{el}}$ for a fixed domain, it follows that $\Delta h_{\min} \sim \frac{1}{N_{\text{el}} k^2}$. Hence the number of time steps N_T grows as $\mathcal{O}(N_{\text{el}} \cdot k^2)$.

2.4 Task 4: "Assembly" of the mass matrix

After discretisation, on each element $T \in \mathcal{T}_h$, the scalar product is pulled back to the reference element $\hat{T} = [-1, 1]^2$ via the geometric mapping $(x, y) = \mathcal{F}^{(T)}(\xi, \eta)$. The mass term therefore yields the global mass matrix $M \in \mathbb{R}^{N_h \times N_h}$ whose entries are defined by: for $i, j = 1, \dots, N_h$,

$$M_{ij} := (\varphi_i, \varphi_j)_{L^2(\frac{1}{\kappa}; \Omega)} = \sum_{T \in \mathcal{T}_h} J^{(T)}(\xi, \eta) (\hat{\varphi}_i, \hat{\varphi}_j)_{L^2(\frac{1}{\kappa}; \hat{T})}, \quad (2.4)$$

where $J^{(T)}(\xi, \eta)$ is the Jacobian determinant of the geometric mapping from the reference element $\hat{T}$ to the physical element T . Each scalar product over $\hat{T}$ is then approximated using the 2D GLL quadrature rule, obtained by tensorising the 1D rule in each reference direction:

$$\int_{\hat{T}} f(\xi, \eta) d\xi d\eta \approx \sum_{p=1}^{\text{nGLL}} \sum_{q=1}^{\text{nGLL}} w_p w_q f(\xi_p, \eta_q), \quad (2.5)$$

where w_p and w_q are the 1D GLL weights acting respectively in the ξ - and η -directions, so that the 2D weight at node (p, q) is simply their product $w_p w_q$. Applying this rule to the mass integral gives:

$$M_{ij} = \sum_{T \in \mathcal{T}_h \ni i} \sum_{p=1}^{\text{nGLL}} \sum_{q=1}^{\text{nGLL}} \frac{1}{\kappa} w_p w_q J_{pq}^{(T)} \hat{\varphi}_i(\xi_p, \eta_q) \hat{\varphi}_j(\xi_p, \eta_q). \quad (2.6)$$

Diagonal (lumped) structure. The key observation that makes the SEM particularly efficient lies in the choice of GLL nodes as both the quadrature points and the interpolation nodes. The local basis functions are tensor products of 1D Lagrange polynomials built on the GLL nodes. Each global node i corresponds, within a given element T , to a unique local position (p, q) via the connectivity table. The basis functions therefore satisfy the cardinality property:

$$\hat{\varphi}_i(\xi_p, \eta_q) = \ell_{p'}(\xi_p) \ell_{q'}(\eta_q) = \delta_{pp'} \delta_{qq'}, \quad (2.7)$$

where (p', q') is the local position of global node i within the element. Each basis function is therefore exactly one at its own node and zero at all others. When substituted into (2.6), the inner product is evaluated at the GLL quadrature points, which coincide with the interpolation nodes. The product $\hat{\varphi}_i \cdot \hat{\varphi}_j$ therefore vanishes at every quadrature point unless $i = j$. All off-diagonal entries are therefore exactly zero — not as an approximation,

but as an algebraic identity. The triple sum in (2.6) collapses to the single term where the quadrature point (p, q) coincides with global node i , giving the diagonal entry:

$$M_{ii} \approx \sum_{T \in \mathcal{T}_h \ni i} \sum_{p=1}^{\text{nGLL}} \sum_{q=1}^{\text{nGLL}} \frac{1}{\kappa} w_p w_q J_{pq}^{(T)} \delta_{pp'} \delta_{qq'} = \sum_{T \in \mathcal{T}_h \ni i} \frac{1}{\kappa} w_{p'} w_{q'} J_{p'q'}^{(T)}. \quad (2.8)$$

where the sum runs over all elements T sharing the global node i , and (p', q') denotes the local position of that node within element T . Here, the diagonal structure is an *exact* consequence of the method's design, and $\mathbf{M}$ reduces to a vector of length N_h assembled by scatter-add over elements.

In practice, the diagonal mass matrix is assembled directly as a global vector. For each element T , the connectivity array maps every local GLL node (p, q) to its global index. The local diagonal contribution

$$M_{pq}^{(T)} = \frac{1}{\kappa} w_p w_q J_{pq}^{(T)} \quad (2.9)$$

is evaluated at the GLL nodes and added to the corresponding global entry through a scatter-add operation. Since the SEM mass matrix is exactly diagonal, no global matrix needs to be formed and the result is stored as a vector of length N_h .

Implement the function `assemble_mass(iglob, jac, wgll, n_elem, nGLL, nglob, kappa)` which returns the diagonal mass matrix as a vector $\mathbf{M}$ of length `nglob`, initialized to zero. The assembly proceeds as follows: for each element $T \in \{1, \dots, N_{el}\}$,

1. Extract the local connectivity slice `iglob(:, :, T)` and the local Jacobian slice `jac(:, :, T)`.
2. Accumulate the local diagonal contribution into the global vector

This scatter-add automatically handles the summation over all elements sharing an interface node, consistently with the expression for M_{ii} derived above.

2.5 Task 5: Time integration

Considering (1.2) we have,

$$\mathbf{M}\ddot{\mathbf{P}}(t) = -\mathbf{K}\mathbf{P}(t) + \mathbf{F}(t). \quad (2.10)$$

Let us introduce the velocity variable $\mathbf{V}(t) = \dot{\mathbf{P}}(t)$ and the acceleration $\mathbf{A}(t) = \mathbf{M}^{-1}(-\mathbf{K}\mathbf{P}(t) + \mathbf{F}(t))$, rewrites as the equivalent first-order system

$$\begin{cases} \dot{\mathbf{P}}(t) = \mathbf{V}(t), \\ \dot{\mathbf{V}}(t) = \mathbf{A}(t). \end{cases} \quad (2.11)$$

For the time integration, we consider the Störmer–Verlet (Leapfrog) scheme which is explicit, second-order accurate, time-reversible, and symplectic (energy conservative). The key idea is to stagger the update of $\mathbf{P}$ and $\mathbf{V}$ around a half-step: starting from $(\mathbf{P}^n, \mathbf{V}^n)$ at time $t^n = n \Delta t$, one step to $t^{n+1} = t^n + \Delta t$ decomposes into four sub-steps:

$$\begin{cases} \mathbf{P}^{n+\frac{1}{2}} = \mathbf{P}^n + \frac{\Delta t}{2} \mathbf{V}^n, \\ \mathbf{A}^{n+\frac{1}{2}} = \mathbf{M}^{-1} \left(-\mathbf{K} \mathbf{P}^{n+\frac{1}{2}} + \mathbf{F}(t^{n+\frac{1}{2}}) \right), \\ \mathbf{V}^{n+1} = \mathbf{V}^n + \Delta t \mathbf{A}^{n+\frac{1}{2}}, \\ \mathbf{P}^{n+1} = \mathbf{P}^{n+\frac{1}{2}} + \frac{\Delta t}{2} \mathbf{V}^{n+1}, \end{cases} \quad (2.12)$$

where $t^{n+\frac{1}{2}} = t^n + \frac{\Delta t}{2}$. Recall that since $\mathbf{M}$ is diagonal, its inversion reduces to a pointwise division. All vectors $\mathbf{P}$, $\mathbf{V}$, $\mathbf{A}$ and $\mathbf{M}$ are of length `nglob` and must be initialized to zero before the time marching scheme. Note that the computation of $(-\mathbf{K} \mathbf{P}^{n+\frac{1}{2}})$ is addressed in Task 6 and the computation of the source term is addressed in Task 7.

Question 2. Explain why the error depends only on the space discretization, for the choosen analytical solution.

2.6 Task 6: Matrix-free computation of the stiffness contribution

Since we consider here a matrix-free formulation, the computation of the stiffness contributions is performed at each time step. The idea is to compute the vector $\mathbf{a} = -\mathbf{KP}$, where the entries are: for $i = 1, \dots, N_h$,

$$(\mathbf{KP})_i := a_h(\varphi_i, p_h) = \sum_{T \in \mathcal{T}_h} (\nabla \varphi_i, \nabla p_h)_{L^2(\frac{1}{\rho}; T)}.$$

Pulling the elemental integrals back to the reference element $\hat{T} = [-1, 1]^2$ and applying the 2D GLL quadrature rule gives:

$$(\mathbf{KP})_i = \sum_{T \in \mathcal{T}_h} \sum_{p=1}^{\text{nGLL}} \sum_{q=1}^{\text{nGLL}} w_p w_q J_{pq}^{(T)} \frac{1}{\rho} \left(\partial_x \varphi_i|_{pq} \partial_x p_h|_{pq} + \partial_y \varphi_i|_{pq} \partial_y p_h|_{pq} \right). \quad (2.13)$$

Physical gradients. The physical gradients are obtained from the reference gradients through the inverse transpose of Jacobian:

$$\nabla_x p_h = (\mathbf{J}^{(T)})^{-T} \nabla_\xi p_h, \quad \mathbf{J}^{(T)} = \begin{pmatrix} \partial_\xi x & \partial_\eta x \\ \partial_\xi y & \partial_\eta y \end{pmatrix}.$$

In component form it gives,

$$\partial_x p_h|_{pq} = \frac{1}{J_{pq}^{(T)}} (\partial_\eta y \partial_\xi p_h - \partial_\xi y \partial_\eta p_h)_{pq}, \quad \partial_y p_h|_{pq} = \frac{1}{J_{pq}^{(T)}} (-\partial_\eta x \partial_\xi p_h + \partial_\xi x \partial_\eta p_h)_{pq}, \quad (2.14)$$

where all metric terms are known from Task 2.

Reference gradients. Let $\mathbf{H}$ be the local 1D derivative matrix (obtained in Task 1) with entries $H_{ij} = \ell'_i(\xi_j)$. Due to the tensor-product structure, differentiation in the ξ -direction corresponds to a left multiplication by $\mathbf{H}^T$, while differentiation in the η -direction corresponds to a right multiplication by $\mathbf{H}$. Thus,

$$\partial_\xi p_h = \mathbf{H}^T \mathbf{P}^{(T)}, \quad \partial_\eta p_h = \mathbf{P}^{(T)} \mathbf{H}, \quad (2.15)$$

where $\mathbf{P}^{(T)}$ gathers the local nodal values.

Weighted physical fluxes. Define the quadrature weight matrix $\mathbf{W}$ with entries $W_{pq} = w_p w_q$. The weighted physical fluxes are

$$\mathbf{G}^x = \frac{1}{\rho} \mathbf{W} \mathbf{J}^{(T)} \partial_x p_h, \quad \mathbf{G}^y = \frac{1}{\rho} \mathbf{W} \mathbf{J}^{(T)} \partial_y p_h. \quad (2.16)$$

These fluxes already include the quadrature weights and metric terms. We then define the pullback to the reference element as

$$\mathbf{G}^\xi = \frac{(\partial_\eta y) \mathbf{G}^x - (\partial_\eta x) \mathbf{G}^y}{J^{(T)}}, \quad \mathbf{G}^\eta = \frac{-(\partial_\xi y) \mathbf{G}^x + (\partial_\xi x) \mathbf{G}^y}{J^{(T)}}. \quad (2.17)$$

Using the cardinality property (2.7), the reference gradient of the test function φ_i at local position (p', q') reduces to

$$\partial_\xi \hat{\varphi}_i|_{pq} = \ell'_{p'}(\xi_p) \delta_{qq'}, \quad \partial_\eta \hat{\varphi}_i|_{pq} = \delta_{pp'} \ell'_{q'}(\eta_q), \quad (2.18)$$

the local stiffness action reduces to the tensor form

$$(\mathbf{KP})^{(T)} = \mathbf{H} \mathbf{G}^\xi + \mathbf{G}^\eta \mathbf{H}^T. \quad (2.19)$$

Implementation. For each element T , we gather the local values $\mathbf{P}^{(T)}$. All intermediate quantities are $\text{nGLL} \times \text{nGLL}$ matrices. Pointwise products are used except for multiplications involving $\mathbf{H}$ or $\mathbf{H}^T$, which are standard matrix products.

The local contribution $\mathbf{A}^{(T)} = \mathbf{H} \mathbf{G}^\xi + \mathbf{G}^\eta \mathbf{H}^T$ is finally scatter-subtracted into the global vector to produce

$$\mathbf{a} = -\mathbf{KP}.$$

2.7 Task 7: Matrix-free assembly of the source term

The volumetric source $f(x, y, t)$ enters the semi-discrete system through the global load vector. In the matrix-free setting, the source contribution is directly accumulated into the vector $\mathbf{a}$, which already contains the stiffness contribution $-\mathbf{K}\mathbf{P}$.

For $i = 1, \dots, N_h$, the weak form contribution reads

$$F_i = \sum_{T \in \mathcal{T}_h} (f(x, y, t^{n+\frac{1}{2}}), \varphi_i)_{L^2(T)}$$

Pulling the integral back to the reference element $\hat{T} = [-1, 1]^2$ and applying the 2D GLL quadrature rule yields

$$F_i = \sum_{T \in \mathcal{T}_h} \sum_{p=1}^{\text{nGLL}} \sum_{q=1}^{\text{nGLL}} w_p w_q J_{pq}^{(T)} f(x_{pq}^{(T)}, y_{pq}^{(T)}, t^{n+\frac{1}{2}}) \varphi_i|_{pq}. \quad (2.20)$$

Using the cardinality property (2.7) of the GLL basis, the test function evaluates to a Kronecker delta at quadrature nodes. Therefore, for a local node (p', q') the elemental contribution reduces to the nodal form

$$F_{p'q'}^{(T)} = w_{p'} w_{q'} J_{p'q'}^{(T)} f(x_{p'q'}^{(T)}, y_{p'q'}^{(T)}, t^{n+\frac{1}{2}}). \quad (2.21)$$

Implementation. First, the source is evaluated at the GLL nodes and at the half time step: $f_{pq}^{n+\frac{1}{2}} = f(x_{pq}, y_{pq}, t^{n+\frac{1}{2}})$. For each element T , we extract the local-to-global map and the Jacobian determinant:

$$\text{ig} = \text{iglob}(:, :, T), \quad J^{(T)} = \text{jac}(:, :, T).$$

All intermediate quantities are $\text{nGLL} \times \text{nGLL}$ arrays and the operations below are performed pointwise. We then form the local contribution (2.21) and scatter-add it into the global vector:

$$\mathbf{a}_{\text{iglob}(p,q,T)} += F_{pq}^{(T)}, \quad \forall p, q \in \{1, \dots, \text{nGLL}\}.$$

2.8 Task 8: L^2 and H^1 error computation

To assess the accuracy of the numerical solution p_h , we define the pointwise error

$$e_h(x, y) := p_h(x, y) - p(x, y, T_f),$$

where p is the analytical solution at the final time T_f . The standard L^2 norm and H^1 semi-norm are pulled back to the reference element $\hat{T} = [-1, 1]^2$ and evaluated using the 2D GLL quadrature:

$$\|e_h\|_{L^2(T)}^2 \approx \sum_{p=1}^{\text{nGLL}} \sum_{q=1}^{\text{nGLL}} w_p w_q J_{pq}^{(T)} (e_h)_{pq}^2, \quad (2.22a)$$

$$|e_h|_{H^1(T)}^2 \approx \sum_{p=1}^{\text{nGLL}} \sum_{q=1}^{\text{nGLL}} w_p w_q J_{pq}^{(T)} \left[(\partial_x e_h)^2 + (\partial_y e_h)^2 \right]_{pq}. \quad (2.22b)$$

The physical derivatives of the error are computed from the reference gradients:

$$\partial_x e_h = \frac{1}{J^{(T)}} \left(\partial_\eta y \partial_\xi e_h - \partial_\xi y \partial_\eta e_h \right), \quad \partial_y e_h = \frac{1}{J^{(T)}} \left(-\partial_\eta x \partial_\xi e_h + \partial_\xi x \partial_\eta e_h \right), \quad (2.23)$$

with the reference gradients obtained via the tensor-product derivative matrices:

$$\partial_\xi e_h = H^T e^{(T)}, \quad \partial_\eta e_h = e^{(T)} H, \quad (2.24)$$

where $e^{(T)}$ collects the nodal error values on element T .

Implementation. For each element T , the local-to-global map and Jacobian are extracted:

$$\text{ig} = \text{iglob}(:, :, T), \quad J^{(T)} = \text{jac}(:, :, T), \quad e^{(T)} = p_h(\text{ig}) - p(x(\text{ig}), y(\text{ig}), T_f).$$

All operations below are performed pointwise on $\text{nGLL} \times \text{nGLL}$ arrays. The local contributions to the L^2 and H^1 norms are computed and summed over all elements.

Question 3. Plot the L^2 and H^1 error norms as a function of the mesh size, and verify that the expected convergence rates are observed for order from 2 to 6. Provide a critical analysis of the results.

3 Exercise 2: Reflection of a Ricker wavelet between two parabolic antennas

The goal of this exercise is to run the SEM solver on an unstructured quadrilateral mesh (generated with the software `Gmsh`), representing a physical domain containing two facing parabolic reflectors. An acoustic pulse is emitted from the focus of one antenna (see Figure 2). The wavefronts reflect back and forth between the

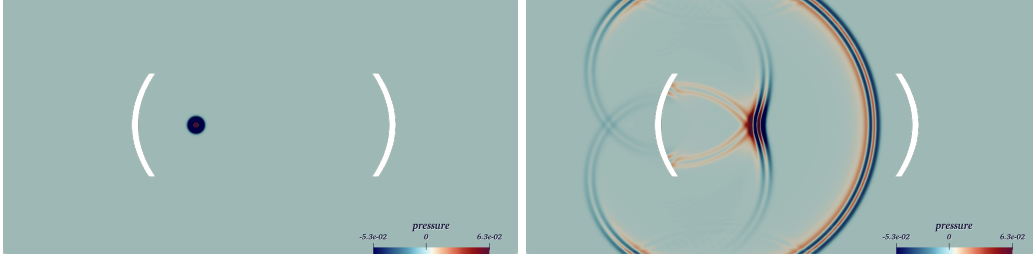


Figure 2: Snapshots of the pressure field at the initial time $t = 0$ s (left frame) and at $t = 1.196$ s (right frame).

two antennas, creating a complex interference pattern. The medium parameters are uniform: $c = 1$, $\rho = 1$, $\kappa = \rho c^2 = 1$, and the solution is computed up to a final time $T_f := 4.0$ s, and the results are visualized as a time series of snapshots showing the pressure field at different time steps.

In the following we only describe the modifications to be made to the code developed in Exercise 1.

3.1 Task 1: Mesh construction from a VTK file

In Exercise 1, the mesh was a structured grid built analytically. Here, the physical mesh is instead read from a provided VTK file, allowing the use of unstructured quadrilateral meshes. A function is provided to handle mesh construction.

Isoparametric mapping. Each quadrilateral element is defined by four corner nodes with physical coordinates $\{(x_k, y_k)\}_{k=1}^4$, ordered counter-clockwise. The physical coordinates of any point $(\xi, \eta) \in [-1, 1]^2$ inside the element are obtained through the standard bilinear mapping:

$$x(\xi, \eta) = \sum_{k=1}^4 N_k(\xi, \eta) x_k, \quad y(\xi, \eta) = \sum_{k=1}^4 N_k(\xi, \eta) y_k, \quad (3.1)$$

where N_k are the bilinear shape functions. Unlike Exercise 1, the Jacobian now varies within each element and must be computed pointwise at every GLL node.

Usage. Call `MeshBoxSEMFromVTK(vtkfile, nGLL, xi)` to construct the SEM mesh, where `vtkfile` is the path to the mesh file, `nGLL` the number of GLL points per direction, and `xi` the GLL nodes on $[-1, 1]$. The function returns:

- `iglob( $i, j, e$ )`: global index of local node (i, j) in element e ,
- `x, y`: physical coordinates of all global nodes,
- `quads`: element connectivity ($4 \times N_{\text{el}}$, corner indices),
- `jac`: Jacobian determinant at each GLL node,
- `dxdxi_all, dx deta_all, dydxi_all, dy deta_all`: metric terms,
- `xs, ys`: raw corner coordinates (for visualization).

3.2 Task 2: Initial condition

The time integration procedure requires the specification of the initial acoustic pressure and velocity fields. In the present test, the initial condition is a smooth and spatially localized pulse centered at (x_c, y_c) .

For $i = 1, \dots, \text{nGLL}$, let (x_i, y_i) be the global nodal coordinates, the squared distance from node i to the pulse center is defined by $r_i^2 := (x_i - x_c)^2 + (y_i - y_c)^2$.

The characteristic width of the pulse is chosen proportional to the horizontal extent of the computational domain, namely $\sigma := 0.005 (\max(x) - \min(x))$.

The initial pressure field is then prescribed as

$$p_i := \left(1 - \frac{r_i^2}{\sigma^2}\right) \exp\left(-\frac{r_i^2}{2\sigma^2}\right), \quad i = 1, \dots, \text{nGLL}, \quad (3.2)$$

which corresponds to a Ricker-type wavelet centered at (x_c, y_c) . The initial velocity field is identically zero throughout the domain, i.e., $v_i := 0$, $i = 1, \dots, \text{nGLL}$. Implement this initial condition.

3.3 Task 3: Absorbing boundary conditions

In order to reduce reflections at the outer boundary of the computational domain, first-order absorbing boundary conditions (ABC) are enforced. To summarize, we consider the following boundary conditions:

- i) to model a perfect reflection of acoustic waves **on the inner boundaries (reflectors)**, we impose homogeneous Neumann boundary conditions,

$$\nabla p \cdot \mathbf{n} = 0 \quad \text{on } \Gamma_{\text{IN}},$$

- ii) to limit reflections **on the external boundary**, we impose an absorbing boundary condition,

$$\partial_t p + c \nabla p \cdot \mathbf{n} = 0 \quad \text{on } \Gamma_{\text{EXT}},$$

where $\mathbf{n}$ is the outward unit normal vector. After semi-discretization, this condition gives rise to a boundary contribution proportional to the nodal velocity.

Question 4. Derive the weak formulation including the absorbing boundary term.

Implementation. The function `precompute_abc` identifies all boundary edges on Γ_{EXT} and returns a structure `abc_data` containing:

- `abc_data.gn`: global node indices on the absorbing boundary,
- `abc_data.coef`: precomputed coefficients $\frac{1}{c} w_j \|J_{\text{edge}}\|$ for each node,
- `abc_data.n_nodes`: total number of boundary node entries.

The coefficient includes the 1D GLL weight w_j along the edge and the boundary Jacobian $\|J_{\text{edge}}\| = \|\partial_\eta \mathbf{x}\|$ (for vertical edges) or $\|\partial_\xi \mathbf{x}\|$ (for horizontal edges), which measures arc-length changes along the boundary.

Implement the function `apply_abc(A, V, abc_data)` which applies the absorbing boundary correction to the acceleration vector. At each time step, this function scatter-subtracts the ABC contribution:

$$\mathbf{A}_{\text{gn}(k)} \leftarrow \text{coef}(k) \cdot \mathbf{V}_{\text{gn}(k)}, \quad k = 1, \dots, \text{n_nodes},$$

where $\mathbf{V} = \dot{\mathbf{P}}$ is the current velocity. This correction must be applied *before* the mass inversion $\mathbf{A} \leftarrow \mathbf{M}^{-1} \mathbf{A}$ in the time-stepping scheme.

3.4 Task 4: Computation of the mechanical energy of the system

The mechanical energy $E(t) = E_{\text{kin}}(t) + E_{\text{pot}}(t)$, of an acoustic medium is defined as the sum of kinetic and potential contributions:

1. The *kinetic energy*, associated with the time variation of the pressure field:

$$E_{\text{kin}}(t) := \frac{1}{2} \int_{\Omega} \frac{1}{\kappa} (\partial_t p(x, y, t))^2 \, dx \, dy.$$

2. The *potential energy*, representing the elastic energy stored in the pressure gradients:

$$E_{\text{pot}}(t) := \frac{1}{2} \int_{\Omega} \frac{1}{\rho} |\nabla p(x, y, t)|^2 \, dx \, dy.$$

Implementation. In the SEM framework, the discrete energies are computed as:

- Kinetic energy: $E_{kin} = \frac{1}{2} \sum_{i=1}^{nglob} M_i v_i^2$, where $v_i = \partial_t P_i$.
- Potential energy: $E_{pot} = -\frac{1}{2} \sum_{i=1}^{nglob} P_i a_{stiff,i}$, where $\mathbf{a}_{stiff} = -\mathbf{KP}$.

3.5 Task 5: Generation of the results and visualization

Using the provided function `save_snapshot(fig_snap, iglob, x, y, p, time(it), n_elem, nGLL, VISU_MODE, snap_dir, OUTit)` generate a time series of snapshots showing the pressure field at different time steps. The snapshots are saved in the directory `snap_dir` with filenames following the pattern `snapshot_it.png`, where `it` is the current iteration number.

Question 5. Run the simulation both with and without the absorbing boundary condition, and compare the results (energy evolution over time and solution snapshots). Provide a critical analysis of the impact of the ABC on solution quality, particularly regarding the amplitude of reflected waves and the overall energy conservation.